

LING 570: Hw1
Due date: 11pm on Oct 8, 2026 (Thu)

Please read the [information-about-assignments.pdf](#) carefully. Remember that \$hwDir for hw1 refers to /mnt/dropbox/26-27/570/hw1/ on patas, and \$hwDir in future assignments is defined similarly (i.e., hw1 is replaced with hw2, hw3, etc.).

Hw1 has four parts:

- Q1: a simple exercise for you to get used to patas.
- Q2-Q5: probability prerequisites for ling570
- Q6-Q7: FSA prerequisites for ling570

Q1 (20 points): Write a shell script, **make_voc.sh**, that creates a vocabulary from the input text.

- The command line should be: `./make_voc.sh <inputfilename> <outputfilename>`
 - Where
 - `<inputfilename>` is the name of the input file
 - `<outputfilename>` is the name of the output file
- “Writing a shell script” means creating a Python program that does the work and then a two-line shell script to call the Python code. An example shell script is under /mnt/dropbox/26-27/570/python-example/.
- The tool reads each line from the input file, breaks the line into tokens by whitespace, keeps track of the frequencies of those tokens, and finally writes out the tokens and their frequencies to the specified output file.
- The output file is of the format “token frequency”, and the two fields are separated by a tab. No need to sort the output as we will sort the lines in your output file first before comparing that with the gold standard.
- Sample input and output files are under \$hwDir/examples/. If you run your program in your working directory as “`./make_voc.sh $hwDir/examples/sample.in ./tmp.out`”, and then run “`sort ./tmp.out ./tmp.out2`”, the output file `./tmp.out2` should be identical to `sample.out`.
- You should also create a condor command file, **hw1.cmd**, which invokes your shell script where
 - `<infilename>`: `$hwDir/examples/data.in` (note the full path here),
 - `<outfilename>`: `./data.out` (note the relative path here)
 - This means that in the `.cmd` file:
 - `executable=./make_voc.sh`, and
 - `arguments ="/mnt/dropbox/26-27/570/hw1/examples/data.in ./data.out"`
- Finally, create a submission file **hw.tar.gz**, a gzipped tar file including all shell scripts, source code, binaries, and any other files required for assignment.
- In your working directory, run the following: `tar -cvzf hw.tar.gz *`

Q2 (5 points): When you flip a coin, the probability of getting a head is 0.8. Now suppose you flip this coin five times. What is the probability of getting AT LEAST four heads out of the five flips? Please write down the formula, in addition to providing the probability.

Q3 (8 points): There are two random variables X and Y , and the joint probability $P(X, Y)$ is shown below:

	$X=0$	$X=1$
$Y=0$	0.50	0.25
$Y=1$	0.10	0.15

- (2 pts) What is the probability distribution for $P(X)$? **Note that a probability distribution is a function, not a number.**
- (2 pts) What is the probability distribution for $P(Y)$?
- (2 pts) What is the probability distribution for $P(Y|X)$?
- (2 pts) Are X and Y independent? Why or why not?

Q4 (10 points): There are three coins: c_1 , c_2 , and c_3 . When tossing a coin once, the probabilities of getting a head for c_1 , c_2 , and c_3 are 0.1, 0.4, and 0.7, respectively. Now suppose that you pick one of the coins, with the probability 0.2 of the coin being c_1 , 0.5 of being c_2 , 0.3 of being c_3 .

- a) (5 pts) Now that you have picked a coin, you toss the coin once, what is the probability of getting a head?
- b) (5 pts) If you toss this selected coin once and get a head, what is the probability that c_1 was the coin selected in the first step? Hint: you can use Bayes' rule to compute the probability.

Q5 (16 points) Let X , Y , and Z be three random variables.

- a) (8 pts) If X and Y are conditionally independent given Z , is it always true that X and Y are independent? If so, give a proof. If not, provide a counterexample: you can provide $P(X, Y, Z)$ or use text to explain what X , Y and Z represent in your example.
- b) (8 pts) If X and Y are independent, is it always true that X and Y are conditionally independent given Z ? If so, provide a proof. If not, provide a counterexample.

Q6 (15 points): Formal grammars and formal languages

- a) (5 pts) Write a regular grammar that generates the language $\{a^*cb^*\}$; a^* means any number of a 's.
- b) (5 pts) Write a context-free grammar that generates the language $\{a^n cb^n \mid n \geq 0\}$; that is, a string in this language will have n a 's, followed by one c , and then n b 's; n is a non-negative integer (e.g., 0, 1, 2, ...).
- c) (5 pts) Write a context-free grammar that generates the language $\{x y \mid x \text{ is a string over } \{a,b,c\}, y \text{ is a reverse of } x\}$.

Q7 (25 points): FSA and FST

- a) (5 pts) Draw a finite state automaton (FSA) that will accept all the strings that start with zero or more “a”, followed by one or more “b”, but nothing else. For instance, the FSA will accept “abb”, “bb”, “aab”, and will reject “aaa”, “aba”, “bab”.
- b) (5 pts) Draw a finite state transducer (FST) that will accept all strings that contain zero or more “a”, followed by one or more “b” or “c”, and the output would replace each “a” in the input string with a “1”, each “b” with “2”, and each “c” with “3”. For instance, the FST will transduce “ab” to “12”, “aabcbbc” to “1123223”. It will reject “a”, “ba”, etc.
- c) (5 pts) Are regular grammar and FSA equivalent? What does equivalent mean in this context? No need to give a proof or counterexample for this question.
- d) (10 pts) Can all regular expressions be represented by an FSA? That is, for any regular expression, does there exist an FSA that determines whether a given string matches the regular expression?
- If the answer is No, please provide a counter example; that is, show a regular expression that cannot be represented by an FSA.
 - If the answer is Yes, think about how you can write an algorithm that takes a regular expression as input, and outputs an FSA that determines whether the input matches the regular expression. Instead of writing pseudo code to show how the algorithm works, draw an FSA for the following regular expression using the algorithm you have in mind (you can use non-deterministic FSA for this):

$(a^* b c^* \mid d e)^* f g^* h$

Note: $(a^* b c^* | d e)$ means $a^* b c^*$ (a string with zero or more a's, followed by one b, and zero or more c's) or the string "de".

Submission instruction:

- Submit `readme.[txt|pdf]` and `hw.tar.gz` as two separate files.
- The note file, `readme.[txt|pdf]`, should include the answers to Q2-Q7 and any notes that you want us to read.
- `hw.tar.gz` should include the following (as specified in **submit-file-list**):
 - **make_voc.sh** and any source code called by it, and
 - **data.out** produced by running your **make_voc.sh** on `patas`, where **data.out** is the name of the outputfile corresponding to `$hwDir/examples/data.in`